Lab 3: GenAI for HRI

INFO 5356: Introduction to Human-Robot Interaction

Nophar Shalom, Wiam Salih
Cornell Tech
{ns2242, ws553}@cornell.edu

1. Introduction

Hands-on building activities such as LEGO are a core part of middle school STEM education, helping students develop spatial reasoning and learn to follow multi-step instructions. They depend on timely feedback, though: a misplaced brick in an early step often goes unnoticed until several steps later, when it is harder to fix, and a single teacher supervising many groups cannot check every student at every step. Students are also often reluctant to ask for help in front of peers. A robot coach that watches a student build and checks each step could close this gap. For HRI, this raises an open question about grounding visual perception in physical, real-world tasks: the robot must judge small spatial details (a brick's color, position, and orientation) from its own camera under real conditions, and communicate its judgments in a way a student will trust, since both false alarms and missed mistakes can undermine the student's confidence in the robot.

Social robots have been widely studied as tutors for children, and a robot that scaffolds its instructions can improve learning outcomes [1]. Recent work adds LLMs to child–robot interaction [2], [3]. In parallel, foundation models are increasingly used to check whether a physical task was done correctly: LLMs and VLMs can detect and explain robot manipulation failures [4], [5], [6], and a VLM has been used to validate each step of a LEGO assembly through an AR headset [7].

These lines of work leave three gaps. First, VLM failure detection mostly checks a *robot's own* actions, not a *human's* work, which brings new challenges such as hands blocking the view and builds that vary between students. Second, child–robot interaction research has mostly focused on conversational tutoring, where the child's activity happens in speech or on a screen rather than with physical objects. Third, the closest step-verification system [7] relies on an AR headset, which is expensive for classrooms and lacks the social cues, such as gaze and gestures, that make a robot feel like a learning partner. These gaps matter because it remains unknown whether today's VLMs are reliable enough to support hands-on learning once they are embedded in a real robot interacting with a person.

We address these gaps with a Reachy Mini-built desktop robot coach (Fig. 1). The robot guides a student through an 8-step LEGO build, checks the piece the student plans to use, photographs each finished step with its head camera, and verifies it with a VLM. It then nods and wiggles its antennas for a correct step, or tilts its head and says what to fix. Rather than evaluating the VLM in isolation, we evaluate the *end-to-end* robot: whether it verifies each step correctly, which mistakes it misses, and how quickly it responds. We also compare three ways of giving the VLM the "correct answer" on identical photos: a reference image of the correct step, written instructions, or a VLM description judged by an LLM.



Fig. 1. LEGO coach. Reachy Mini guides a student through an 8-step build. After each step it photographs the build with its head camera and checks it with a vision-language model, then nods and wiggles its antennas for a correct step, or tilts its head and says what to fix. We evaluate how accurately this end-to-end loop catches building mistakes, and whether giving the VLM a reference photo of the correct step helps.

Our work addressed the following **research questions**:

- **RQ1:** How accurately does the robot coach verify each building step, and which kinds of student mistakes does it miss most often?
- **H2:** The robot will verify steps more accurately when the VLM gets a reference image of the correct step than when it gets only a text description of that step.

Video demo: Google Drive Link

Code: GitHub Repository

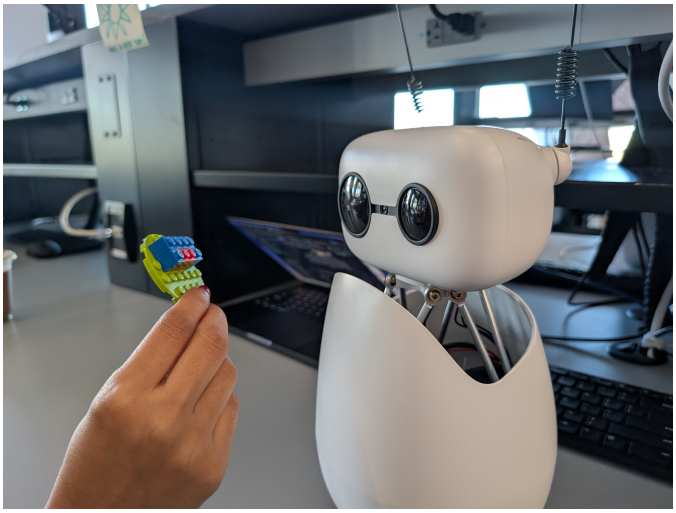


Fig. 2. Reachy Mini judging step completion of user in the study.

2. Background

A. Research theme 1

B. Social robots and generative AI for children's learning

Social robots have been widely studied as tutors for children. Ligthart et al. [1] showed in a three-session study with 130 children that a robot math tutor improved learning when it scaffolded its instructions. More recently, LLMs have been added to child–robot interaction: Rudenko et al. [2] examined the language quality of an LLM-powered robot for preschoolers and stressed that generative output must be validated before it reaches children, and Fernandes et al. [3] found that classroom learning programs are limited by small staff and mixed-ability groups, the same supervision problem found in hands-on STEM classrooms. **Gap:** in this work the robot supports learning through conversation, and the child's activity happens in speech or on a screen. Robots that watch and give feedback on a child's *physical*, hands-on work have rarely been studied. Our robot coach fills this gap by checking a student's LEGO build after every step.

C. Research theme 2

D. Vision-language models for task verification

Foundation models are increasingly used to check whether a physical task has succeeded. REFLECT [4] summarizes a robot's multisensory observations into text so an LLM can explain and correct failures, and AHA [5] trains a VLM to detect manipulation failures and explain them in natural language. Guardian [6] notes that such detectors are limited by scarce failure data. Close to our task, Kottom et al. [7] validate each step of a LEGO assembly with a VLM and provide feedback through an AR headset. **Gap:** REFLECT, AHA, and Guardian check a *robot's own* actions, not a human's work, which brings new problems, such as hands blocking the view and builds that differ between students. The closest LEGO system [7] uses an expensive headset that loses the social cues of an embodied partner. It is also unclear how a VLM should be given the "correct answer." We evaluate a low-cost desktop social robot that verifies each step of a student's build, and we compare three ways of giving the VLM the target: a reference photo, written instructions, or a VLM description judged by an LLM.

3. Robot Control Architecture

Goals and system-focused problem. The robot acts as a desktop building coach. It guides a student through an 8-step LEGO build, checks the work after every step, and gives immediate spoken and gestural feedback so mistakes are caught before the student moves on. The system-focused problem is *VLM-based visual verification of a human's physical work*: the robot must judge the color, size, position, and orientation of small bricks from its own head camera, under real conditions such as partial occlusion by fingers and changing viewpoints, and respond quickly enough to keep the interaction flowing.

Design motivation. Following Ligthart et al. [1], the robot scaffolds the build into one instruction at a time and adds a piece check before each step, so that picking the wrong brick is caught before it is built in. Following Rudenko et al. [2], the robot speaks feedback only after a structured correct/incorrect decision, and every decision is logged against ground truth. Unlike the headset in [7], feedback comes from an embodied robot whose gaze, nods, head tilts, and antennas make its attention and judgment visible. Because failure data are scarce [6], the models are used zero-shot, and we compare three ways of presenting the target, motivated by findings that how observations are presented shapes a foundation model's judgment [4], [5]. We assess the goals with objective end-to-end measures: verification accuracy, missed-mistake rate, false-alarm rate, missed mistakes by type, and response latency (Sec. 4-D).

Inputs. (1) Images from Reachy Mini's head camera, looking forward at a piece the student holds up or down at the build; (2) the build definition (the piece and instruction for each step); (3) reference photos of the correct build after each step, taken from the robot's own camera; and (4) keyboard input, where Enter signals that a piece is ready or a step is built.

Outputs. (1) Speech: the instruction, piece confirmation, and feedback (praise or what to fix), using OpenAI text-to-speech on the robot's speaker or `pyttsx3` with local models; (2) head motion: looking at the piece or build, a nod for a correct step, and a tilt for a mistake; (3) an antenna wiggle for a correct step; and (4) a log entry and photo for every check.

Modules (Fig. 3):

1) **Step Manager** (`lego_coach.py`, `steps.py`): walks through the 8 steps in order, speaking each instruction and running a piece check, then a build check.
2) **Camera & Head Control** (`common.py`): moves the head within the course safety limits and captures one frame.

3) **Piece Identifier** (VLM): names the held-up piece and checks it against the expected piece; on a mismatch, the robot asks for the right one.
4) **Build Verifier** (VLM): compares the build photo with the reference photo (`vlm_image`) or the written instructions (`vlm_text`) and returns a verdict and a feedback sentence.
5) **Build Describer + Judge** (`llm_judge`): the VLM only lists the pieces it sees and where; an LLM that never sees the image compares this description with the instructions and decides.
6) **Feedback Behavior**: a correct step gets a nod, an antenna wiggle, and praise; a mistake gets a head tilt and a spoken hint.
7) **Trial Logger**: writes one row per check (verdict, raw model output, ground truth, mistake type, latency) to `trials.csv`, with the photo in `captures/`.

With `--api`, the course's OpenAI model (`gpt-6.1-sol`) serves as both VLM and LLM; otherwise Gemma 3 4B and Llama 3.2 run locally via Ollama. *[Confirm which models the reported results used.]*

Architecture type. The architecture is *deliberative*, *closed-loop*, and *event-driven*. It is deliberative because each decision follows a perceive–reason–act cycle: the robot captures an image, a VLM (and in `llm_judge`, an LLM) reasons about it against a model of the correct step, and only then does the robot act. It is closed-loop because the feedback changes the student's next action: a flagged mistake is fixed and re-checked before the build moves on. This design fits the task. Building happens in discrete steps, so the robot only needs to judge the build at the end of each step. Accuracy also matters more than reaction speed, since a missed mistake or a false alarm harms a learner more than a few seconds' wait. A reactive design such as color thresholds could not reason about position, orientation, or instructions, and an open-loop design would never verify the step at all. Turn-taking by keypress prevents judging a build while the student's hands are still moving.

4. Evaluation

A. Experimental Design

Independent variable. The verification condition has three levels: `vlm_image` (VLM + reference photo), `vlm_text` (VLM + written instructions), and `llm_judge` (VLM describes, LLM decides). We also stage the build type to break down results: a correct build, or one with a wrong color, missing piece, wrong position, or rotated piece.

Dependent variables. Verification accuracy, missed-mistake rate, false-alarm rate, missed mistakes by type, response latency, and, for the live sessions, piece-identification accuracy.

Design. We use a within-subjects design in which the unit of comparison is the build photo. The live sessions run the robot's default `vlm_image` condition (RQ1). Afterwards, every photo is re-checked offline under all three conditions (H2), so each condition sees exactly the same images, lighting, and mistakes. This pairing removes photo-to-photo differences that would otherwise confound a small study, and it allows an exact McNemar test for each pair of conditions. Condition order needs no counterbalancing because conditions are applied offline and the models do not learn between checks. Instead, we counterbalance mistakes: each participant makes one of each mistake type, at different steps per participant (Latin square). RQ1 is answered by the live accuracy, missed-mistake, and false-alarm rates and the missed mistakes by type. H2 is supported if `vlm_image` is more accurate than `vlm_text` on the same photos. `llm_judge` shows whether separating perception from reasoning helps (Fig. 4).

B. Study Trials

Participants. Four participants (*[age range]*) were recruited by convenience sampling from team members and classmates. Because the study measures the robot's verification performance rather than learning, adult participants suit this system-focused evaluation, and their differing hands and building styles test perception robustness.

Trials. *[Update if your plan changes.]* Each participant completes the 8-step build once and makes four planned mistakes, each of which is checked, fixed, and re-checked. This gives 8 piece checks and 12 build checks per participant. In total there are 32 piece checks and 48 build checks (32 correct, 16 incorrect, 4 of each mistake type), plus 144 replay judgments (48 photos $\times$ 3 conditions).

Data collection and analysis. All data come from inference; no model is trained. Each check is logged to `trials.csv` (participant, step, check type, condition, verdict, raw VLM/LLM output, ground truth, mistake type, latency) with its photo in `captures/`, and the replay is logged to `replay.csv`. Sessions are video recorded. `analyze.py` computes each measure per condition (latency as mean $\pm$ SD), counts missed mistakes by type, and runs exact McNemar tests between conditions ($\alpha = 0.05$, treated as exploratory given the sample size). We also read the raw model output for every wrong verdict to explain the failure.

C. Study Task

The participant sits at a desk facing Reachy Mini. At each step, the robot says which piece is needed and what to do with it. The participant holds the piece up to the camera and presses Enter, and the robot names the piece and either confirms it or asks for the right one. The participant then builds the step and presses Enter. The robot looks down, photographs the build, and verifies it. If the step is correct, the robot nods, wiggles its antennas, and confirms the step. If it is wrong, the robot tilts its head and says what to fix. At four steps listed on a card, the participant makes a planned mistake, then fixes it and has it re-checked. Human inputs are nonverbal (holding up pieces, building, pressing Enter), and robot outputs are verbal (instructions, feedback) and nonverbal (gaze, nod, tilt, antennas).

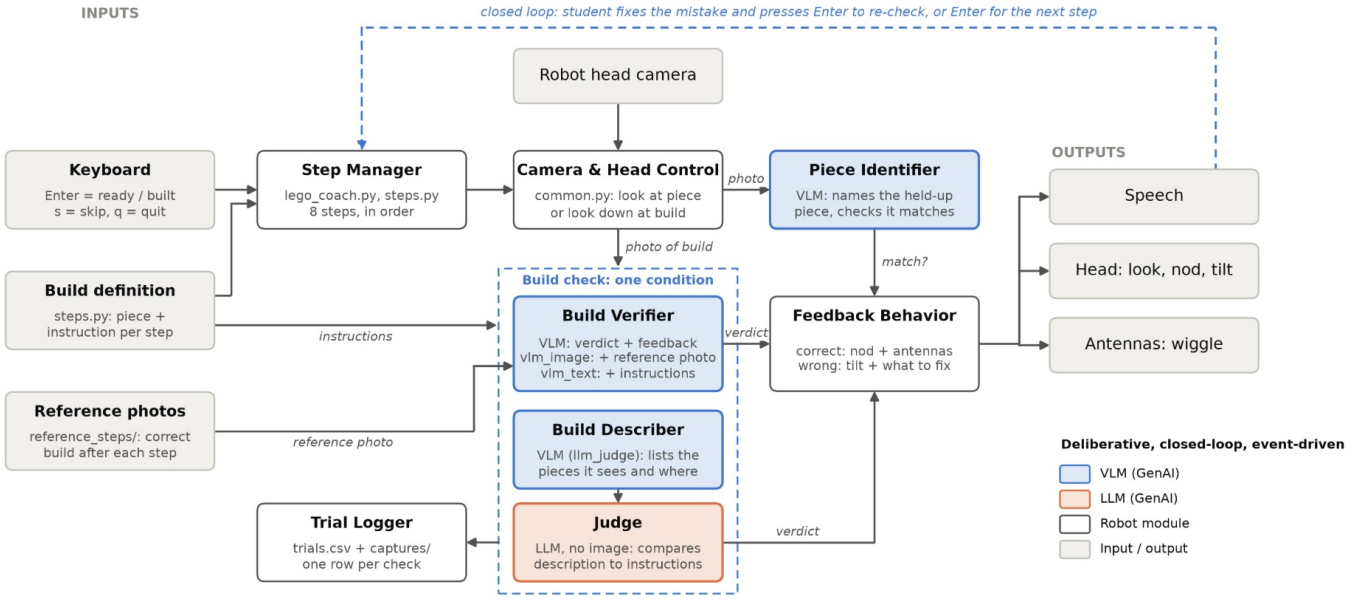


Fig. 3. Robot control architecture. At each step, the robot checks the piece the student holds up, then photographs the finished build and verifies it in one of three ways: a VLM with a reference photo (`vlm_image`), a VLM with written instructions (`vlm_text`), or a VLM that describes the build and an LLM that judges it (`llm_judge`). The robot responds with speech and gestures, and the student fixes any mistake before it is re-checked, closing the loop.

Autonomy. The robot is semi-autonomous, with a human in the loop only for timing. Enter signals that a piece or step is ready, as a stand-in for automatic "done" detection. Everything after that runs autonomously: head motion, image capture, VLM/LLM verification, and feedback. After the robot responds, the experimenter types the ground truth (correct or not, and the mistake type). This is used only for logging and never changes the robot's behavior. A check can be skipped if the piece is out of frame (*[number skipped]*).

D. Metrics

For each build check, the ground truth is either correct or incorrect. **Verification accuracy** is the share of checks where the robot's verdict matches the ground truth, the main measure in VLM failure-detection work [5], [6], [7]. The **missed-mistake rate** is the share of incorrect builds judged correct; it is the most harmful error for a coach, because the student moves on with a mistake built in. The **false-alarm rate** is the share of correct builds judged incorrect, which wrongly corrects a student and can erode trust in the robot [2]. **Missed mistakes by type** report the missed-mistake rate separately for wrong color, missing, wrong position, and rotated pieces, answering the second part of RQ1. **Response latency** is the time in seconds from the Enter press to the robot's feedback (mean ± SD). **Piece-identification accuracy** is the share of held-up pieces named correctly.

5. Results

Live robot (RQ1). The live robot judged 28 of 50 build checks correctly (56.0%; per participant 57.5 ± 13.1%). It approved 5 of the 10 incorrect builds, a missed-mistake rate of 50%: both missing-piece builds, both rotated-piece builds, and 1 of 3 wrong-position builds. It caught both wrong-step builds, the wrong-color build, and 2 of 3 wrong-position builds (Table II). It also rejected 17 of the 40 correct builds (false-alarm rate 42.5%), three of them because it could not see the build clearly. False alarms clustered at steps 5–7,

TABLE I

BUILD VERIFICATION. THE LIVE ROW IS THE ROBOT AS IT RAN; THE REPLAY ROWS JUDGE THE SAME 50 PHOTOS UNDER EACH CONDITION. ACCURACY IS POOLED AND PER PARTICIPANT (MEAN ± SD); MISSED = INCORRECT BUILDS APPROVED (OF 10); FALSE ALARMS = CORRECT BUILDS REJECTED (OF 40); LATENCY IN SECONDS (MEAN ± SD).

Condition	Acc. (%)	Acc. per P	Missed	False al.	Latency
Live `vlm_image`	56.0	57.5 ± 13.1	5/10	17/40	8.3 ± 5.8
`vlm_image`	62.0	63.6 ± 16.2	3/10	16/40	9.1 ± 5.9
`vlm_text`	**68.0**	**67.4 ± 7.8**	4/10	12/40	**8.3 ± 4.9**
`llm_judge`	22.0	24.1 ± 19.8	**0/10**	39/40	21.5 ± 8.2

TABLE II

MISSED MISTAKES BY TYPE (INCORRECT BUILDS APPROVED / INCORRECT BUILDS OF THAT TYPE).

Condition	Missing	Rotated	Position	Wrong step	Color
Live `vlm_image`	2/2	2/2	1/3	0/2	0/1
`vlm_image`	2/2	1/2	0/3	0/2	0/1
`vlm_text`	0/2	1/2	3/3	0/2	0/1
`llm_judge`	0/2	0/2	0/3	0/2	0/1

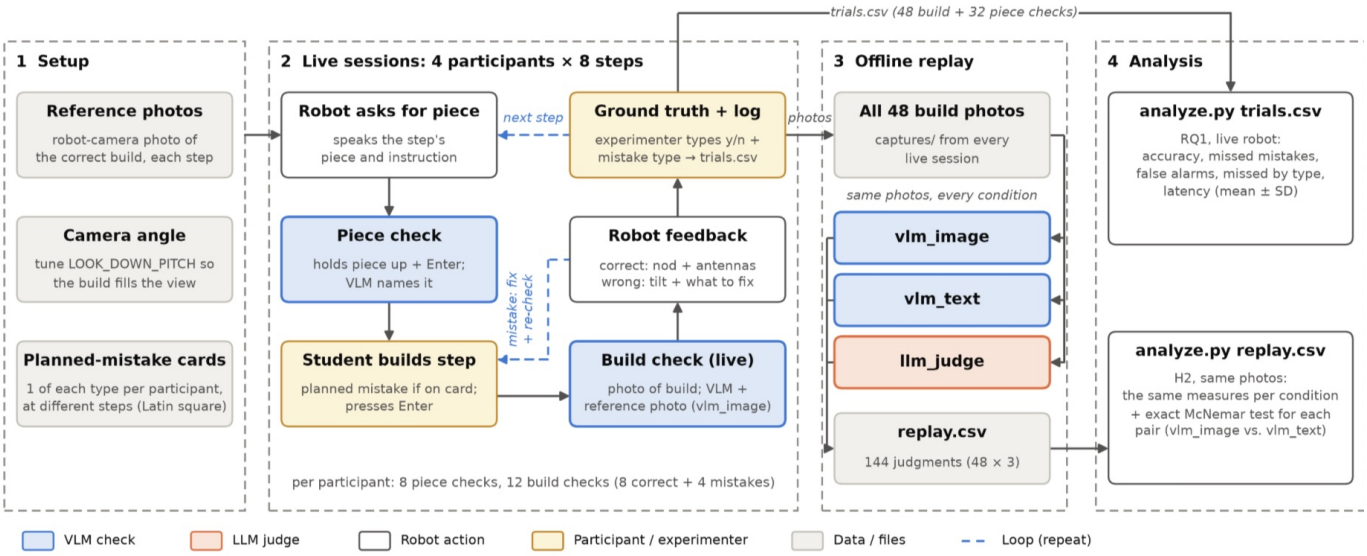


Fig. 4. Evaluation workflow. Four participants each complete the 8-step build with the live robot and make four planned mistakes, one of each type, which they fix and re-check. After every check, the experimenter records the ground truth. All build photos are then re-checked offline with all three verification conditions, so the conditions are compared on identical images.

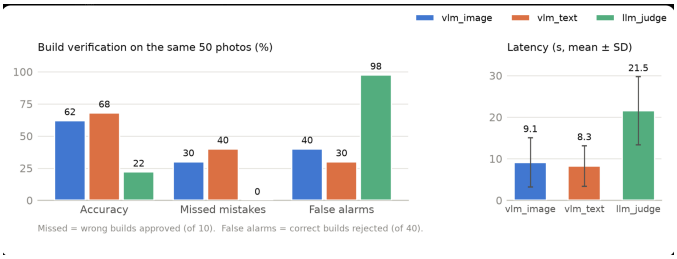


Fig. 5. Replay results on the same 50 build photos. Written instructions (`vlm_text`) were most accurate, the reference photo (`vlm_image`) missed fewer mistakes but raised more false alarms, and `llm_judge` rejected almost every build and was slowest.

where the black angled roof pieces are added (14 of 17), and varied widely by participant (1 to 9). Piece checks were more reliable. The robot named 37 of 45 pieces correctly (82.2%; per participant $83.2 \pm 7.2\%$), accepted none of the 6 wrong pieces, and rejected 8 of 39 correct pieces, half of them by asking to see the piece closer. Build checks took 8.28 ± 5.84 s, rising from 5.12 ± 1.43 s at steps 1–4 to 10.76 ± 6.77 s at steps 5–8. Piece checks took 3.80 ± 1.46 s.

Conditions on identical photos (H2). On the same 50 photos, `vlm_text` was the most accurate condition (68.0%), followed by `vlm_image` (62.0%) and `llm_judge` (22.0%) (Table I, Fig. 5). For `vlm_image` vs. `vlm_text`, the reference photo was right where the text was wrong on 4 photos, and the reverse on 7 (exact McNemar $p = 0.55$). Both VLM conditions outperformed `llm_judge` (23 vs. 3 and 27 vs. 4 discordant photos, both $p < 0.001$). Re-running `vlm_image` on the photos from the live session changed its verdict on 7 of 50 photos (86% agreement).

Patterns. The two VLM conditions traded error types: `vlm_image` missed fewer mistakes (3 vs. 4 of 10) but raised more false alarms (16 vs. 12 of 40). They also missed different mistakes. `vlm_image` missed both missing-piece builds, which `vlm_text` caught, while `vlm_text` missed all three wrong-position builds, which `vlm_image` caught. `llm_judge` approved only 1 of the 50 photos. It never missed a mistake, but it rejected 39 of 40 correct builds. Its VLM description called the photo blurry or unclear in 36 of 50 cases, and the text-only LLM would not approve a build it could not confirm. It was also the slowest condition (21.5 ± 8.2 s).

6. Discussion

A. *Can a robot coach catch a student's mistakes? (RQ1)*

Not reliably yet. End to end, the robot judged just over half of the build steps correctly, and its errors went both ways. It missed half of the mistakes, including every missing-piece and rotated-piece build, and in each of those cases it praised the student while the mistake stayed in the build. These mistakes are subtle from the robot's viewpoint: a missing brick under the blue 2×4 is hidden, and an angled roof piece turned around looks similar from above. In one case the robot described a rotated piece as having its "slope facing outward, matching the reference." In contrast, it caught mistakes that change the overall shape or color of the build, such as a wrong-color brick or a build from the wrong step. This matches the open problem in prior work: VLMs can verify task steps, but fine-grained spatial details remain difficult [5], [6]. False alarms were the more common error (17 vs. 5) and clustered at the roof steps, where the robot repeatedly asked participants to turn pieces that were already correct; P02 was rejected four times in a row

at step 5. For a coach, both errors are costly. A missed mistake carries forward, while repeated wrong corrections can frustrate a student and erode trust in the robot [2]. The piece check, by contrast, was 82% accurate and never accepted a wrong piece. This suggests that naming one isolated, held-up object is much easier for the VLM than judging small spatial relations inside an assembly. In answer to RQ1, the robot verified 56% of build steps correctly and most often missed missing and rotated pieces.

B. How should the VLM be given the correct answer? (H2)

H2 was not supported. The reference image did not improve accuracy: written instructions were numerically better (68% vs. 62%), and the difference was not significant ($p = 0.55$). The two conditions failed in complementary ways. The reference photo caught every wrong-position build that the text missed, while the text caught both missing-piece builds that the photo missed. A photo gives a concrete spatial target for *where* pieces go, whereas a written piece list makes it explicit *which* pieces must be there, so the two forms of the answer appear to be complementary rather than interchangeable. Separating perception from reasoning (`llm_judge`) failed. This condition mirrors REFLECT's approach of summarizing observations into text before reasoning about them [4]. In our setting, however, the VLM's text summary of a blurry, hand-held build mostly reported uncertainty, and the LLM, unable to see the image, rejected nearly everything while taking more than twice as long. Uncertainty and errors in perception carried straight through to the verdict. Finally, the same `vlm_image` condition changed its verdict on 14% of photos between the live run and the replay. Differences of a few photos between conditions are therefore within the model's run-to-run variability.

C. Implications for HRI design

Compared with the AR headset of Kottom et al. [7], our results show that step-by-step VLM verification can be delivered through a low-cost embodied robot with natural spoken hints and feedback in about 8 s. However, accuracy is not yet high enough for a student to rely on the robot without a teacher. Three design takeaways follow. First, give the VLM *both* a reference photo and the written piece list, since each caught mistakes the other missed. Second, image quality was a major bottleneck: blur, hand-held builds, and partial views drove many false alarms. A fixed build platform or a second camera angle, and a robot that asks for a better view rather than forcing a verdict, are likely to help more than a different prompt. Third, smaller checks work better: the piece check was far more accurate than the whole-build check. Our contributions are an end-to-end robot coach that checks a person's physical work with a VLM and responds with social gestures, an evaluation showing which building mistakes such a robot catches or misses, and a paired comparison of three ways to give a VLM the target on identical photos.

7. Future Work & Limitations

Small adult sample with few mistakes. Only four adult team members and classmates took part, rather than middle school students, and their mistakes were planned rather than accidental. The number of mistakes also varied by participant (0 to 5), leaving only 10 incorrect build photos. Our missed-mistake rates therefore rest on 1–3 photos per mistake type, and with 50 photos the McNemar tests can only detect large differences between conditions. The vlm_image vs. vlm_text result should be read as "no clear difference," not as evidence that the two are equal. Children handle pieces less steadily than adults and make mixed, accidental errors, so performance with the target users may be lower still.

One build, one setting, manual trigger. We tested one 8-step build at one desk under one lighting condition. The reference photos were taken from the same camera position used in the study, and we tested a single model (`gpt-6.1-sol`), run once per photo and condition, even though its verdicts changed on 14% of photos between runs. The participant pressed Enter to start each check, so the robot never decided by itself when a step was finished. Finally, ground truth was typed live by the experimenter. Four inconsistent entries were corrected before the replay, and a few labels may still be wrong.

Future work. Next steps are to test with middle school students in a classroom, measuring learning, engagement, and trust alongside accuracy [1]. Others are to replace the Enter key with automatic detection of a finished step (for example, a gesture, or hands leaving the camera view), to have the robot look from a second angle before judging rotated or hidden pieces, and to test more builds, models, and lighting conditions.

References

[1] M. Ligthart, S. de Droog, M. Bossema, L. Elloumi, K. Hoogland, M. Smakman, K. Hindriks, and S. Ben Allouch, "Design specifications for a social robot math tutor," in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2023, pp. 321–330.

[2] *[Full title and authors from Google Scholar.]* Rudenko et al., LLM-powered social robot for preschoolers, in *Companion of the ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2026.

[3] N. Fernandes *et al.*, "Co-designing a social robot for newcomer children's cultural and language learning," in *Proc. ACM/IEEE Int. Conf. Human-Robot Interaction (HRI)*, 2026.

[4] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," in *Proc. Conf. Robot Learning (CoRL)*, PMLR 229, 2023, pp. 3468–3484.

[5] J. Duan *et al.*, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[6] P. Pacaud, R. Garcia, and S. Chen, "Guardian: Detecting robotic planning and execution errors with vision-language models," in *CoRL Workshop*, 2025.

[7] Kottom, Gharib, Humml, and Stefan-Zavala, "Augmented reality and vision-language models to guide humans across manual tasks," in *SIGGRAPH Immersive Pavilion*, 2025.